Common Functions Sheet

The thirteen functions in `shared/confio.py`: what each one does, what it returns, and which tools carry a copy of it.

13 FUNCTIONS | 11 TOOLS | 52 COPIES CHECKED

SUMMARY · READ-ONLY REVIEW

What is in shared/confio.py

Code that more than one tool needs is written once, in `shared/confio.py`, and then copied unchanged into each tool that uses it. The file's `USED_BY` table lists every function and the tools that carry a copy.

1.1 The copy rule

- Change the reference in `shared/confio.py` first.
- Copy the changed function into every tool listed for it in `USED_BY`. A tool's own wrapper may call the copy.
- `ci/check_repo.py` fails any tool whose copy differs from the reference.
- Tool files carry no comments or docstrings (a repo rule), so each copy should match its reference exactly.

1.2 Parity check for this sheet

Each of the 13 functions was compared with every copy listed in `USED_BY`: 52 copies in 11 tool files. Each comparison matched the reference structurally (syntax tree, ignoring comments). No copy differed and no listed copy was missing.

The repository clone used for this sheet has no `origin/main` branch, so `ci/check_repo.py` itself was not run. Run it before any pull request that touches `shared/confio.py`.

1.3 Where the functions are used

TOOL (FILE PREFIX)	COPIES	GROUP
nodecontrol (full build)	2	Files, allmon3_sync
nodecontrol_pi02w	5	Files, firewall, allmon3_sync
sysmon (full build)	1	Files
sysmon_pi02w	4	Files, firewall
nodetalker	10	Files, Asterisk, firewall, allmon3_sync, suitelog
amp_nodetalker	4	Files, firewall
44helper	8	Files, config, network, firewall, system, allmon3_sync
wifimon	5	Files, firewall, system
instmon	7	Firewall, suitelog
confputter	2	Files, config
pibrary	4	Files, firewall

Counts come from the `USED_BY` table. A firewall function counts once for each of its three names. The copies add to 52.

The thirteen functions

Grouped by what they do. The last column gives the number of tools that carry a copy.

FUNCTION	GROUP	WHAT IT DOES	TOOLS
atomic_write	Files	Writes a file safely: a temporary file, then an atomic replace.	10
fill_secrets	Config	Puts saved secrets back into a config file that holds placeholders.	2
http_get	Network	Fetches an https address with a size limit.	1
firewall_backend	Firewall	Reports which firewall is active: firewalld, ufw or none.	8
open_firewall_port	Firewall	Opens a port in the active firewall.	8
close_firewall_port	Firewall	Closes a port the tools opened.	8
ami_credentials	Asterisk	Finds the AMI login in manager.conf.	1
allmon3_sync	Asterisk	Adds or removes node sections in allmon3.ini.	4
system_profile	System	Describes the board, OS, architecture and Wi-Fi manager.	2
suitelog_redact	Suitelog	Hides secrets and tags addresses in a log line.	2
suitelog_append	Suitelog	Adds one row to a log file, with locking and rotation.	2
suitelog_rows	Suitelog	Reads log rows back, oldest first, optionally filtered by kind.	2
suitelog_flag	Suitelog	Reads a one-line flag file.	2

Writing and filling files

3.1 `atomic_write(path, data, mode=None, new_mode=0o600, skip_same=False)`

Returns True when the file was written, False when it was skipped.

- Writes to a temporary file in the same folder, created with mode 0600, then syncs it to disk and replaces the target in one step. A reader sees the old file or the new one, never a part of each.
- When run as root and the file already exists, keeps the file's owner and group.
- Mode: the explicit `mode` if given, else the existing file's mode, else `new_mode` (0600 by default).
- `skip_same=True` returns False and writes nothing when the content is already identical.
- On any error the temporary file is removed and the error is raised again.

3.2 `fill_secrets(text, current, supplied=None, is_json=False)`

Returns a tuple: the filled text, the count of placeholders resolved, and a list of unresolved placeholders.

- Placeholders look like `<redacted:xxxxxxxx>`. Each one is filled from `supplied` when given, otherwise from the value at the same spot in `current` (the file already on disk), matched by section and line shape.
- A placeholder with no value is returned in the unresolved list. Comment lines (starting with `;` or `#`) are not reported.
- `is_json=True` walks the parsed JSON instead of the lines and returns the JSON text, pretty-printed with sorted keys.

3.3 `http_get(url, max_bytes, timeout=20)`

Returns the response bytes. Only `https://` addresses are allowed; anything else raises ValueError. The request sends the User-Agent `asl-dvs`. A response larger than `max_bytes` raises ValueError rather than being cut short.

Opening and closing ports

The three firewall helpers share one pattern. They look at the firewall that is actually running, change it only when needed, and report what happened with a short code.

4.1 `firewall_backend()`

Returns `"firewalld"` when `firewall-cmd --state` says running, `"ufw"` when `ufw status verbose` says active, otherwise `"none"`. firewalld is checked first. Missing commands and timeouts count as "none".

4.2 `open_firewall_port(port, proto="tcp")` and `close_firewall_port(port, proto="tcp")`

Returns a tuple: a code and a message.

CODE	MEANING IN OPEN	MEANING IN CLOSE
+	Port opened (firewalld: now and after a reboot)	(not used)
-	(not used)	Port closed
=	Already open, or no firewall running	Was not open, or no firewall running
!	Could not open the port	Could not close the port, or ufw's default still allows incoming traffic

- **firewalld:** open treats the port as open if the zone target is ACCEPT or the port is listed. It adds the port both now and permanently. Close removes it both ways and checks again.
- **ufw:** open looks for an ALLOW rule or the default-allow setting, then runs `ufw allow`. Close runs `ufw --force delete allow` and warns when the default still allows incoming traffic.
- With no firewall running, both functions change nothing and return "=".

Asterisk, system profile and the suitelog

5.1 `ami_credentials(conf_path, user="", secret="")`

Returns a (user, secret) pair. If both are given they are returned unchanged. Otherwise the function reads the Asterisk manager file and returns the first section, other than `[general]`, that has a secret. Returns ("", "") when nothing is found. Only Node Talker uses it.

5.2 `allmon3_sync(text, add=(), remove=(), own="")`

Returns the new allmon3.ini text and a list of notes.

- Removes the sections named in `remove`.
- Adds the sections named in `add` (in numeric order). Each new section copies its host, port, user and pass lines from a public node section. Public sections are numbered outside 1000 to 1999, and the `own` node is preferred.
- If there is no public section to copy from, nothing is added and a note asks for a manual change.

5.3 `system_profile(root="/")`

Returns a dictionary: board (`pi` or `other`), model, os_id, os_name, codename, version_id, arch, wifi_manager (NetworkManager, wpa_supplicant, systemd-networkd or unknown), wifi_iface, and wifi_ifaces. The `root` argument lets a test point the function at a copy of the filesystem.

5.4 The suitelog functions

FUNCTION	RETURNS AND BEHAVIOUR
`suitelog_redact(text, salt)`	Returns the text with secrets hidden: Bearer tokens and password, psk, secret, token, key, auth and cookie values become `[hidden]`. SSIDs, MAC addresses and public IP addresses become a kind tag and a four-character salted hash. Loopback and unspecified addresses are kept.
`suitelog_append(path, row, salt=None, seg_max=10000000)`	Adds a millisecond time `t` when missing, writes one JSON line under a file lock, and redacts the line when a salt is given. When the file reaches `seg_max` bytes (10 MB by default) it moves to `path.1`, replacing any earlier one. Returns True when it rotated.
`suitelog_rows(path, kinds=None)`	A generator of rows from `path.1` then `path` (older first). Lines that are not valid JSON are skipped. `kinds` keeps only rows of those kinds.
`suitelog_flag(path)`	Returns the first 64 characters of the file, stripped, or "x" if the file is empty. Returns None when the file cannot be read.

Points for the book

6.1 Single-user functions

`ami_credentials` is used only by Node Talker, and `http_get` only by 44helper.

6.2 Full build and pi02w

The firewall functions are listed for the pi02w tools and the shared tools only. The full-build Node Controller and SysMon do not carry them. Any change to a firewall function therefore has to be copied into the pi02w tools, and the full build is unaffected unless it is listed.

6.3 Return conventions

- Firewall helpers return a code and a message. The codes are +, -, = and !.
- Writers return True or False. `atomic_write` returns False only when `skip_same` found nothing to change.
- `fill_secrets` and `allmon3_sync` return the new text along with what they resolved or noted, so the caller decides what to show.

6.4 How this sheet was made

Read from the source files in the repository on branch ccr-9d4fe19f-ssud83. The sheet describes the code as it stands. It was not written from the tools' manuals.